



STAGE SGSI

-

CRÉATION D'UN CATALOGUE DES APIs

Cahier des charges

Auteur :

KHEIRALLAH Cédric

Superviseur :

MINNE Frédéric

Dernière révision :

10 juillet 2026

1 Introduction

1.1 Objectifs et Public Cible

Nous souhaitons créer un catalogue des APIs disponibles pour les consultants externes au SGSI ainsi que pour les développeurs internes permettant de connaître leurs intérêts pour certaines APIs.

1.2 Contraintes

- Le catalogue doit rester en lecture seule pour les utilisateurs finaux qui ne peuvent pas tester les APIs.
- L'application doit être documentée tant au niveau de son architecture et organisation qu'au niveau du code et de son utilisation.
- Ce projet se fera de manière itérative semaines par semaine tout au long du stage (méthode Agile).

2 Fonctionnalités et Exigences du Système

2.1 Exigences Fonctionnelles

— Must have :

- La page de catalogue doit pouvoir afficher toutes les APIs actuellement mises à disposition des utilisateurs dans l'ordre alphabétique.
- Le tri par défaut doit se faire par ordre alphabétique sur le nom des APIs.
- Pour chaque API, la page de catalogue doit afficher son nom, sa version, sa révision, sa date de sa dernière mise à jour ainsi que son icône (ou un texte de remplacement si indisponible).
- Le catalogue doit être muni d'une barre de recherche permettant d'explorer les APIs par nom.
- L'utilisateur doit pouvoir accéder à la page de documentation d'une API en cliquant sur son nom dans le catalogue.
- La page de documentation doit afficher toutes les informations disponibles sur l'API telles qu'elles sont définies dans l'API Manager.
- La page de documentation doit offrir la possibilité à l'utilisateur de voir les documents et URLs supplémentaires à la documentation de l'API.
- Les "server URLs" présents dans la documentation de l'API doivent apparaître dans leur version correcte.
- L'utilisateur doit pouvoir télécharger le fichier de description d'une API aisément depuis la page de documentation de l'API.
- Le fichier de description téléchargé doit contenir tout les détails déjà présent sur la page de documentation de l'API.
- Le fichier de description téléchargé doit être au format standard de description d'API OpenAPI.
- Les administrateurs doivent pouvoir exclure des APIs.
- Les administrateurs doivent pouvoir consulter les données des APIs stockées dans le répertoire de stockage des données.

- Les administrateurs doivent pouvoir supprimer un répertoire d'API dans le répertoire de stockage des données.
- Les administrateurs doivent pouvoir configurer les paramètres de l'application.
- Les données concernant les APIs doivent se mettre à jour automatiquement à intervalle régulier.
- Les administrateurs doivent pouvoir forcer la mise à jour des données d'une API.
- Les administrateurs doivent pouvoir consulter les logs montrant les erreurs du programme, les connections des administrateurs ainsi que les modifications qu'ils ont effectués.
- Les logs doivent être descriptifs et pouvoir montrer aux administrateurs l'origine de chaque message présent dans les logs (back-end, front-end, scheduler).
- Should have :
 - Le catalogue doit être muni d'une liste de tags permettant de filtrer les APIs.
 - Le catalogue doit offrir la possibilité à l'utilisateur de trier les APIs par leur nom et leur date de dernière mise à jour.
- Could have :
 - Les administrateurs doivent pouvoir faire remonter les statistiques d'utilisation de l'application via Matomo.
- Won't have :
 - L'utilisateur doit pouvoir faire une demande d'accès à une API depuis la page de catalogue et/ou depuis la page de documentation de l'API.
 - La demande d'accès doit pouvoir être reçue par les administrateurs de la plateforme afin d'être traitée et acceptée/refusée.

2.2 Exigences Techniques

- Must have :
 - L'application doit être maintenable.
 - L'application doit être très bien documentée tant au niveau de son architecture et organisation qu'au niveau du code et de son utilisation.
 - L'application doit être isolée dans des conteneurs docker.
 - L'application doit être capable de se mettre à jour automatiquement à intervalle régulier sans aucune intervention.
 - L'application doit stocker les fichiers de méta-données et de documentation des APIs sous format YAML.
 - L'application doit stocker les variables d'environnement sous format texte ou TOML.
 - L'application doit suivre la licence open source MIT.
- Should have :
 - L'application doit utiliser des technologies connues au SGSI.
 - L'application doit utiliser le framework Django pour l'interface web.
 - L'application doit utiliser le langage Python pour la logique en back-end.
 - L'application doit utiliser le scheduler cron pour la planification automatique.
 - L'application ne doit pas dépendre d'un service externe au SGSI.
- Could have :

- L'application doit respecter la charte graphique de l'UCLouvain.
- L'application doit pouvoir faire remonter des statistiques d'utilisation via Matomo en local.
- Won't have :
 - ...

3 Processus de Développement

- S4 : implémentation dans les conteneurs docker + cron setup
- S5 : importeur + extracteur (URL)
- S6 : interface page d'accueil fonctionnelle (dirige vers les pages d'APIs)
- S7 : variables d'environnement
- S8 : (matomo)
- S9 : faire joli + doc
- S10 : démo

4 Glossaire

- **API** : Application Programming Interface ou Interface de Programmation d'Application est un ensemble normalisé de classes, de méthodes, de fonctions et de constantes qui sert de façade par laquelle un logiciel offre des services à d'autres logiciels.
- **API Manager** : Outil de gestion des APIs permettant de les créer, de les publier, de les sécuriser et de les surveiller.
- **API Controller** : Outil en ligne de commande permettant de communiquer avec API Manager
- **Application** : l'application complète au sens général du terme, composée de 3 conteneurs docker : "back-end", "front-end" et "scheduler"
 - **Back-end** : le back-end de l'application qui contient l'extracteur, l'importeur et le répertoire de stockage des données
 - **Extracteur** : l'extracteur de l'application qui s'occupe de récupérer les données des APIs depuis API Manager en utilisant l'API Controller, de les extraire de l'archive et de les passer à l'importeur
 - **Importeur** : l'importeur de l'application qui s'occupe de traiter et préparer les données pour le front-end et de les stocker dans le répertoire de stockage
 - **Répertoire de stockage des données** : le répertoire de stockage des données de l'application qui contient les données d'API traitées par l'importeur et prêtes à être utilisées par le front-end ainsi que toutes les données utiles pour faire fonctionner l'application
 - ▷ **Données d'API** : fichier de méta-données (api.yaml), fichier de documentation (swagger.yaml) et documents attachés (plain text)
 - ▷ **Configurations** : variables d'environnement, logs, blacklist, etc...
 - **Front-end** : le front-end de l'application qui contient l'interface utilisateur
 - **Page de catalogue des APIs** : la page de catalogue de l'application qui affiche la liste des APIs disponibles à l'utilisateur
 - **Page de documentation d'une API** : la page de documentation d'une API de l'application qui affiche les informations détaillées d'une API à l'utilisateur
 - **Scheduler** : le scheduler de l'application qui s'occupe de régénérer automatiquement le catalogue des APIs à intervalle régulier